

0.75.2. Arrivé en (0, 0), le A devient B — doublé, deux positions

Deux A, deux départs, la même case d'arrivée : chacun devient B en arrivant en (0, 0).

1. Ce qu'il faut comprendre

C'est le 0.75 et le 0.75.1 réunis : deux A, l'un parti de (10, 0), l'autre de (10, 8). La même croix les déplace ensemble.

Chacun a ses variables : `x`, `y`, `lettre` pour le premier ; `xb`, `yb`, `lettreB` pour le second. Et chacun a **sa condition**, sur **sa** position, mais vers la **même** case : (0, 0).

Ils n'y arrivent pas en même temps. Avec la flèche gauche, le premier arrive en (0, 0) et devient B ; le second, en (0, 8), reste un A. Avec la flèche haut, le premier ne peut plus monter (il est déjà en haut) ; le second monte, arrive en (0, 0), et devient B à son tour.

À la fin, les deux B sont sur la même case, (0, 0) : on n'en voit qu'un. Chaque lettre ne connaît que sa propre position ; rien ne les empêche de se retrouver au même endroit.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1930 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 10;          // le premier A : départ (10, 0)
```

2. UNE VARIABLE — Y

```
uint8_t y = 0;
```

3. UNE VARIABLE — LETTRE

```
uint8_t lettre = 0;
```

4. UNE VARIABLE — XB

```
uint8_t xb = 10;         // le second A : départ (10, 8)
```

5. UNE VARIABLE — YB

```
uint8_t yb = 8;
```

6. UNE VARIABLE — LETTREB

```
uint8_t lettreB = 0;
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[lettre], 250);    // le premier
        deplace_croix(xb, yb, ALPHABET[lettreB], 250); // le second, dans la même image
        nombre(0, 17, x);                             // la position du premier
        nombre(4, 17, y);
        nombre(10, 17, xb);                             // celle du second
        nombre(14, 17, yb);

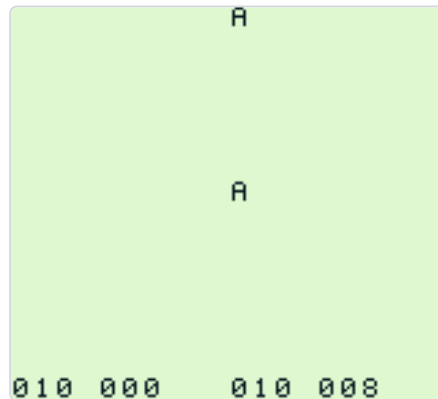
        // La même case pour les deux : (0, 0). Chacun regarde SA position.
        // 1. Le premier devient B quand il arrive en (0, 0).
        // Sur une ligne : if (x == 0 && y == 0) lettre = 1;
        // Avec des accolades, la même chose :
        if (x == 0 && y == 0) {
            lettre = 1;
        }
        // 2. Le second aussi, quand il y arrive à son tour.
        // Sur une ligne : if (xb == 0 && yb == 0) lettreB = 1;
        // Avec des accolades, la même chose :
        if (xb == 0 && yb == 0) {
            lettreB = 1;
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Deux A ; à gauche, le premier arrive en (0, 0) et devient B ; en montant, le second y arrive aussi et devient B.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ à gauche : le premier est B, le second encore A

✓ en montant, le second devient B en (0, 0)

✓ un B en (0, 0)

Ensuite : **0.76. Un déclencheur en (0, 0) : actif** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++